# Method Overriding

23 March 2024 18:27

If subclass(child class) has the same method declaration in the parent class, it is known as **method overriding in Java.**

**What are the usage of method overriding?**

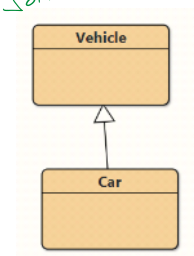
- Method overriding is used to provides the specific implementation of the method which is already provided by its superclass.
- Method overriding is used for runtime polymorphism.

**Rules for method overriding**

- The method must have the same signature as in the parent class.
- There must be IS-A relationship between the class or interfaces.

```
package Polymorphism;
public class Vehicle
{
   public void run(){
      System.out.print("\nVehicle is running.");
   }
}
```

signature's are same




dynamic memory
Space allocated at runtime of code.

```
package Polymorphism;
public class Car extends Vehicle
{
   @Override
   public void run(){
      super.run();    //reusing the parent class code by calling its method.
      System.out.print("\nCar is running smoothly.");
   }
}
```

code reusability.

memory → heap.

created during compilation. → class area.

all static method & variables reside

all the objects are residing & non-static methods.

**Can we override static method? Why?**

No, static method cannot be overridden.

It is because the static method is bound with class whereas instance method is bound with an object. Static belongs to the class area, and an instance belongs to the heap area.

**Can we override Java main method?**

No, because the main is a static method.

**Difference between method Overloading and Overriding in Java**

| **Method Overloading** | **Method Overriding** |
|---|---|
| Method overloading is used to increase the readability of the code. | Method overriding is used to provide specific implementation of the method that is already provided by its super class. |
| Method overloading is performed inside the class. | Method overring occurs in two classes that have IS-A relationship(Inheritance). |
| In case of method overloading, parameter must be different. | In case of method overriding, parameter must be same. |
| Method overloading is the example of compile time polymorphism. | Method overriding is the example of runtime polymorphism. |

**Covariant return type**

Return type may vary in the same direction as the subclass. Earlier version of Java is not supporting this type of feature.
Covariant return types have been (partially) allowed in the Java language since the release of JDK5.0,[2] so the following example wouldn't compile on a previous release.

```
public class Vehicle
{
   private int engineNum;
   public Vehicle(int en){
      this.engineNum = en;
   }

   public void run(){
      System.out.print("\nVehicle is running.");
   }

   public Vehicle getVehicle(){
      return new Vehicle(engineNum); //returning the object of Vehicle.
   }
}

public class Car extends Vehicle
{
   Car(int engineNum){
      super(engineNum);
   }
   @Override
   public void run(){
      super.run();    //reusing the parent class code by calling its method.
      System.out.print("\nCar is running smoothly.");
   }

   @Override
   public Car getVehicle(){
      return new Car(12345); //returning the object of Vehicle.
   }
}
```

Return type is child class type. (or covariant type).

**Advantages of Covariant return type**

- Covariant return type assist to stay away from confusing typecast in the class hierarchy and makes the code more usable, readable and maintainable.
- In method overriding, the covariant return type provides the liberty to have more to the point return types.
- Covariant return type helps in preventing the run-time ClassCastExceptions on returns.

**Super keyword in Java**

The super keyword in Java is a reference variable which is used to refer immediate parent class. Whenever you create the instance of subclass, an instance of parent class is created implicitly which is referred by **super** reference variable.

**Usage of super keyword**

- **super keyword** can be used to refer immediate parent class instance variable.
- **super keyword** can be used to invoke immediate parent class method.
- **super()** can be used to invoke immediate parent class constructor.

**Instance initializer block**

This block is used to initialize the instance data member. It run each time when object of the class is created.

The initialization of the instance variable can be done directly but there can be performed extra operations while initializing the instance variable in the instance initializer block.

**What is invoked first, instance initializer block or constructor?**

Instance initializer block is invoked at the time of object creation. The Java compiler copies the instance initializer block in the constructor after the first statement super(). So firstly constructor is invoked.

**Rules for instance initializer block:**

- The instance initializer block is created when instance of the class is created
- The instance initializer block is invoked after the parent class constructor is invoked(i.e. after the super() constructor call).
- The instance initializer block comes in order in which they appear.

**final Keyword in Java**

The **final** keyword in Java is used to restrict the user. The Java final keyword can be used in many context.

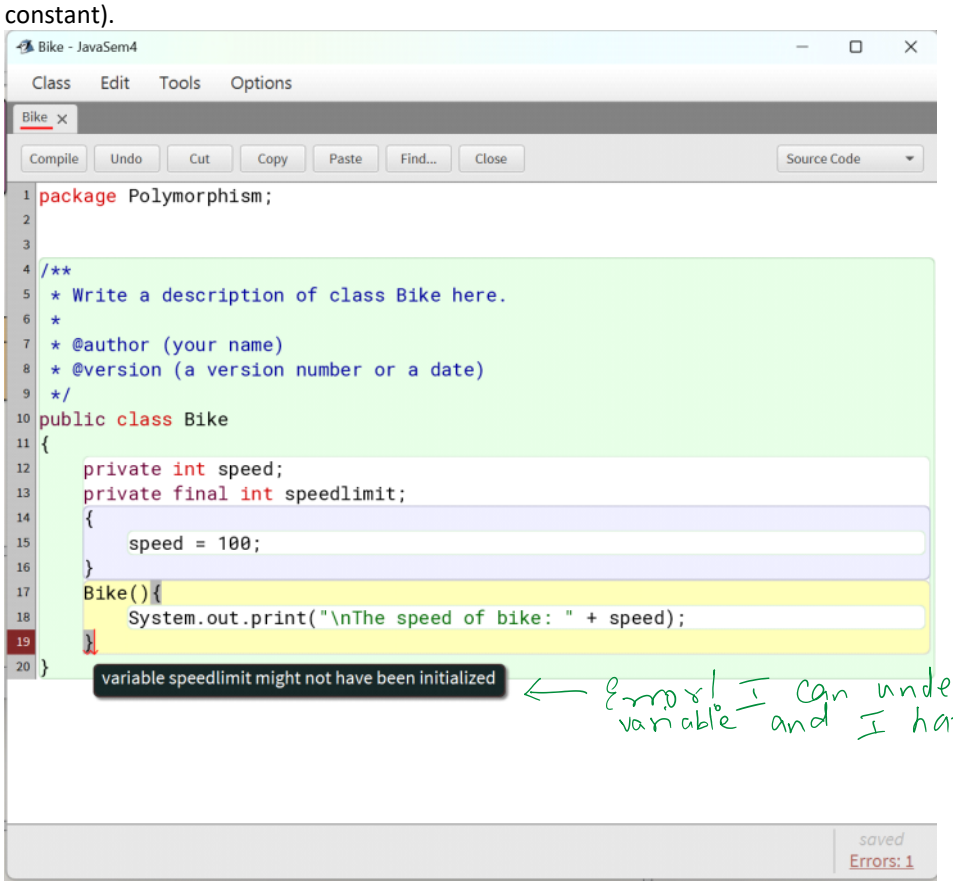
Final can be:

1. Variable
2. Method
3. Class

The **final** keyword can be applied with the variables, a final variable that have no value it is called blank final variable or uninitialized final variable. It can be initialized in the constructor only (if it is not initialized). The blank final variable can be static also which will be initialized in the static block only.

1. **Java final Variable**

   If you make any variable as final, you cannot change the value of the final variable(it will be a constant).




← Error! I can understand that I declared final variable and I haven't initialized it.

## 2. Java final method

If you make any method as final, you cannot override it.

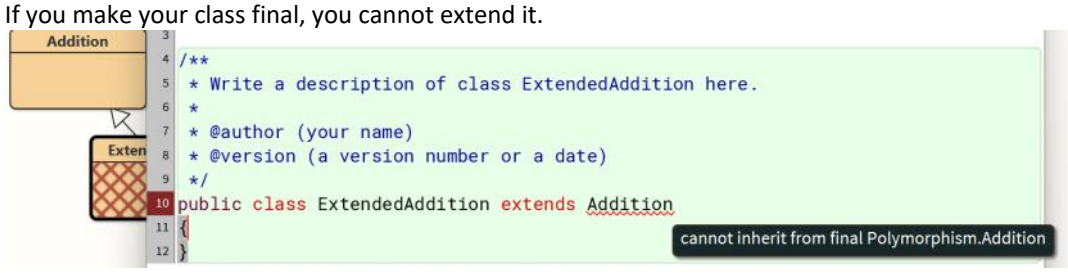
```
/**
 * Write a description of class Car here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Car extends Vehicle
{
    Car(int engineNum){
        super(engineNum);
    }
    @Override
    public void run(){
        //super.run()
        System.out.pr
    }

    @Override
    public Car getVehicle(){
        return new Car(12345); //returning the object of Vehicle.
    }
}
```

run() in Polymorphism.Car cannot override run() in Polymorphism.Vehicle
overridden method is final

Java compiler will not allow us.

## 3. Java final class

If you make your class final, you cannot extend it.




**Q. Is final method inherited?**

Yes, final method is inherited but you cannot override it.

**static blank final variable**

A static final variable that is not initialized at the time of declaration is known as static blank final variable. It can be initialized only in static block.

```
public class A
{
   static final double PI;
   static{
      PI = 22.0/7;
   }

   public static void main(String args[]){
      System.out.print("\fPI = "+ PI);
   }
}
```

**Q. What is final parameter?**

If you declare any parameter as final, you cannot change the value of it.

```
public class A
{
    static final double PI;
    static{
        PI = 22.0/7;
    }
```

```
    static final double PI;
    static{
        PI = 22.0/7;
    }
    public void printSumOfDigits(final int num){
        int s = 0;
        for(;num > 0; s = s + (num % 10), num/= 10);
        System.out.print("\nSum of digits: "
    }
    public static void main(String args[]){
        System.out.print("\fPI = "+ PI);
    }
}
```

final parameter num may not be assigned

**Q. Can we declare a constructor final?**

No, because constructor is never inherited.

```
*/
public class A
{
    final A(){
        |
    }
```

modifier final not allowed here